LEMBAR KERJA PRAKTIKUM

MODUL 5 — STRUKTUR PERULANGAN: for, while, dan do-while

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Bahasa Pemrograman	C
Nama Mahasiswa	Muhammad Rassya Abhirama
NIM / Kelas	2610511046 / B

Petunjuk Penggunaan LKP

Lampirkan screenshot source code dan screenshot output/terminal pada tempat yang disediakan. Untuk Debugging Challenge, lampirkan bukti sebelum perbaikan, pesan error/warning, source code setelah perbaikan, dan output akhir.

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Menampilkan Deret Menggunakan for

Kerjakan sesuai Modul 5 dan simpan sebagai latihan_01.c.

Screenshot Source Code

latihan_01.c > main(void)

```c
#include <stdio.h>

int main(void)
{
    int batas;
    int i;

    printf("Masukkan batas akhir: ");
    scanf("%d", &batas);

    if (batas < 1)
    {
        printf("Batas harus minimal 1.\n");
        return 1;
    }

    printf("Deret bilangan:\n");

    for (i = 1; i <= batas; i++)
    {
        printf("%d ", i);
    }

    printf("\n");

    return 0;
}
```

Screenshot Output Program

```
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\output> & .\'latihan_01.exe'
Masukkan batas akhir: 10
Deret bilangan:
1 2 3 4 5 6 7 8 9 10
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\output>
```

Pertanyaan analisis

1. Apa nilai awal counter?

Jawaban = 1

2. Apa kondisi perulangannya?

Jawaban = (i <= batas)

3. Apa bentuk update counter?

Jawaban = i + 1 atau i++ dalam kode

4. Berapa jumlah iterasi untuk input 5?

Jawaban = 5

5. Apa hasil apabila kondisi diubah menjadi i < batas?

Jawaban = hanya akan menampilkan deretan dari 1 hingga batas (dikurangi satu)

6. Apa hasil apabila nilai awal diubah menjadi 0?

Jawaban = akan menampilkan deretan angka dari nol

7. Apa hasil apabila update diubah menjadi i += 2?

Jawaban = akan menampilkan bilangan ganjil dari 1 hingga batas

8. Mengapa input kurang dari 1 ditolak?

Jawaban = karena kita telah menginisiasikan i = 1

2. Praktikum Terbimbing 2 — Menghitung Jumlah dan Rata-Rata

Kerjakan sesuai Modul 5 dan simpan sebagai latihan_02.c.

Screenshot Source Code

latihan_02.c > main(void)

```c
#include <stdio.h>

int main(void)
{
    int banyak_data;
    int i;
    double nilai;
    double total = 0.0;
    double rata_rata;

    printf("Masukkan banyak data: ");
    scanf("%d", &banyak_data);

    if (banyak_data <= 0 )
    {
        printf("Banyak data harus lebih besar dari nol.\n");
        return 1;
    }

    for (i = 1; i <= banyak_data; i++)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%lf", &nilai);

        total += nilai;
    }

    rata_rata = total / banyak_data;

    printf("\nHASIL\n");
    printf("Banyak data : %d\n", banyak_data);
    printf("Total : %.2f\n", total);
    printf("Rata-rata : %.2f\n", rata_rata);

    return 0;

}
```

Screenshot Output Program

```
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\output> & .\'latihan_02.exe'
Masukkan banyak data: 5
Masukkan nilai ke-1: 70
Masukkan nilai ke-2: 90
Masukkan nilai ke-3: 50
Masukkan nilai ke-4: 80
Masukkan nilai ke-5: 68

HASIL
Banyak data : 5
Total : 358.00
Rata-rata : 71.60
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\output>
```

Pengujian tambahan

Test Case	Banyak Data	Data Nilai	Total	Rata-rata
1	4	70,80,90,60	300	75
2	5	70,90,50,80,60	350	70
3	6	20,60,50,70,45,75	320	53.33

Pertanyaan analisis

1. Mengapa total diinisialisasi dengan 0.0?

Jawaban = untuk menyatakan bahwa nilai total pada awal program Adalah nol, untuk proses counter pada line selanjutnya

2. Apa fungsi total += nilai?

Jawaban = untuk langsung menjumlahkan semua nilai yang diinput

3. Mengapa banyak_data tidak boleh nol?

Jawaban = jika banyak data nol, maka tidak ada data yang bisa diinput

4. Berapa kali perulangan berjalan apabila banyak data adalah 4?

Jawaban = 4 kali

5. Apa yang terjadi apabila nilai total tidak diinisialisasi?

Jawaban = tidak terjadi apa apa

6. Apakah rata-rata mebutuhkan casting?

Jawaban = tidak perlu karna dalam rumus rata-rata total sudah tipe data double

7. Mengapa dalam program ini casting tidak wajib?

Jawaban = karena salah satu data dalam pembagian rata-rata Adalah double

3. Praktikum Terbimbing 3 — Input Berbasis Sentinel Menggunakan while

Kerjakan sesuai Modul 5 dan simpan sebagai latihan_03.c.

Screenshot Source Code

latihan_03.c > main(void)

```c
#include <stdio.h>

int main(void)
{
    double nilai;
    double total = 0.0;
    double rata_rata;
    double nilai_tertinggi = 0.0;
    double nilai_terendah = 0.0;

    int jumlah_data = 0.0;
    int jumlah_lulus = 0.0;

    printf("Masukkan nilai 0-100, gunakan -1 untuk berhenti.\n");
    printf("Nilai: ");
    scanf("%lf", &nilai);

    while (nilai != -1.0)
    {
        if ((nilai < 0.0) || (nilai > 100.0))
        {
            printf("Nilai tidak valid dan tidak diproses.\n");
        }
        else
        {
            if (jumlah_data == 0)
            {
                nilai_tertinggi = nilai;
                nilai_terendah = nilai;
            }
            else
            {
                if (nilai > nilai_tertinggi)
                {
                    nilai_tertinggi = nilai;
                }
                if (nilai < nilai_terendah)
                {
                    nilai_terendah = nilai;
                }
            }

            total += nilai;
            jumlah_data++;

            if (nilai >= 60.0)
            {
                jumlah_lulus++;
            }
        }

        printf("Nilai: ");
        scanf("%lf", &nilai);
    }

    if (jumlah_data > 0)
    {
        rata_rata = total / jumlah_data;

        printf("\nHASIL REKAP\n");
        printf("Jumlah data      : %d\n", jumlah_data);
        printf("Total nilai      : %.2f\n", total);
        printf("Rata-rata        : %.2f\n", rata_rata);
        printf("Nilai tertinggi  : %.2f\n", nilai_tertinggi);
        printf("Nilai terendah   : %.2f\n", nilai_terendah);
        printf("Jumlah lulus     : %d\n", jumlah_lulus);
    }
    else
    {
        printf("\nTidak ada data valid yang diproses.\n");
    }

    return 0;

}
```

Screenshot Output Program

```
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrogram
an\Module_05\output> & .\'latihan_03.exe'
Masukkan nilai 0-100, gunakan -1 untuk berhenti.
Nilai: 90
Nilai: 100
Nilai: 75
Nilai: 85
Nilai: 90
Nilai: -1

HASIL REKAP
Jumlah data      : 5
Total nilai      : 440.00
Rata-rata        : 88.00
Nilai tertinggi  : 100.00
Nilai terendah   : 75.00
Jumlah lulus     : 5
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrogram
an\Module_05\output>
```

Pengujian input sentinel

Test Case	Urutan Input	Jumlah Data	Total	Rata-rata	Jumlah Lulus	Catatan
1	90,100,75,85,90	5	440	88	5	Tertinggi = 100, terendah = 75
2	75,50,80,45,60,90,70	7	470	67.14	5	Tertinggi = 90, terendah = 45
3	97,60,55,80,0,95,90,87	8	564	70.50	6	Tertinggi = 97, terendah = 0

Pertanyaan analisis

1. Mengapa input pertama dilakukan sebelum while?

Jawaban = karena untuk kondisi bisa dijalankan, harus ada data input terlebih dahulu

2. Mengapa input kembali dilakukan di akhir blok?

Jawaban = supaya program tidak terhentikan, dan tetap bisa menginput nilai

3. Apakah nilai -1 ikut dihitung?

Jawaban = tidka, dan program akan berhenti dieksekusi

4. Mengapa jumlah_data diperiksa sebelum menghitung rata-rata?

Jawaban = supaya tidak ada pembagian nol

5. Mengapa nilai tertinggi dan terendah diinisialisasi dari data pertama?

Jawaban = karena nilai input akan tersimpan di data nilai_tertinggi dan nilai_terendah secara terpisah, dan akan terus tersimpan menyesuaikan dengan input yang baru

6. Apa yang terjadi apabila pengguna langsung memasukkan -1?

Jawaban = program berhenti dan menampilkan "tidak ada data yang valid"

7. Apa yang terjadi apabila pengguna memasukkan 120?

Jawaban = nilai 120 tidak akan tersimpan ke data nilai

8. Apakah 120 menambah jumlah_data?

Jawaban = tidak

9. Berapa jumlah lulus jika data adalah 50, 60, 70, dan 80?

Jawaban = ada 3 yaitu 60,70,80

4. Praktikum Terbimbing 4 — Validasi Input Menggunakan do-while

Kerjakan sesuai Modul 5 dan simpan sebagai latihan_04.c.

Screenshot Source Code



latihan_04.c > main(void)

```c
#include <stdio.h>

int main(void)
{
    int jumlah_barang;

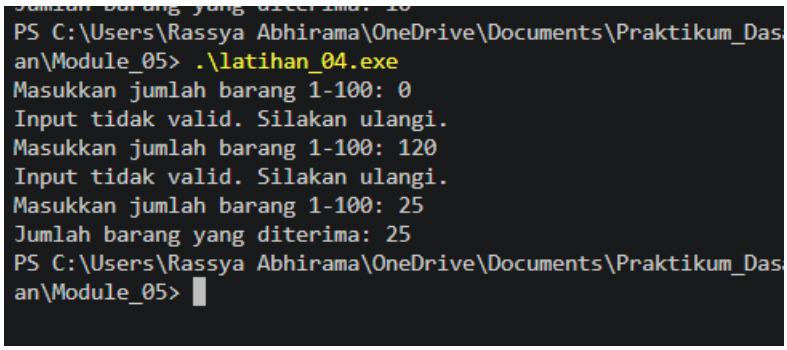
    do
    {
        printf("Masukkan jumlah barang 1-100: ");
        scanf("%d", &jumlah_barang);

        if ((jumlah_barang < 1) || (jumlah_barang > 100))
        {
            printf("Input tidak valid. Silakan ulangi.\n");
        }
    }
    while ((jumlah_barang < 1) || (jumlah_barang > 100));

    printf("Jumlah barang yang diterima: %d\n", jumlah_barang);

    return 0;
}
```

Screenshot Output Program



Pertanyaan analisis

1. Mengapa do-while tepat untuk kasus ini?

Jawaban = karena do-while akan mengulangi blok jika nilai benar, dan jika salah program akan tereksekusi sekali

2. Berapa kali blok dijalankan minimal?

Jawaban = 1 kali

3. Apa kondisi agar perulangan terus berjalan?

Jawaban = jika blok benar

4. Apa kondisi agar perulangan berhenti?

Jawaban = jika blok salah, atau nilai salah

5. Mengapa terdapat titik koma setelah while?

Jawaban = untuk menyatakan kondisi terakhir pada program

6. Apa akibatnya apabila kondisi diubah menjadi &&?

Jawaban = kondisi while tidak akan terbaca, dan nilai input yang salah pun akan terus terbaca program ke blok lainnya

7. Dapatkah program ini dibuat menggunakan while?

Jawaban = tidak, karena akan terus terulang jika blok benar

V. Eksperimen dan Tracing

1. Latihan Tracing

Kerjakan berdasarkan tracing.c pada Modul 5. Jangan langsung menjalankan program. Lengkapi trace table terlebih dahulu.

Iterasi	i	total sebelum	total sesudah	i % 2 == 0	jumlah_genap
1	1	0	1	-	-
2	2	1	3	0	1
3	3	3	6	-	1
4	4	6	10	0	2
5	5	10	15	-	2
6	6	15	21	0	3

Prediksi nilai akhir i:

Jawaban = 6

Prediksi nilai total:

Jawaban = 21

Prediksi nilai jumlah_genap:

Jawaban = 3

Screenshot Source Code tracing.c

```c
#include <stdio.h>

int main(void)
{
    int i;
    int total = 0;
    int jumlah_genap = 0;

    for (i = 1; i <= 6; i++)
    {
        total += i;

        if (i % 2 == 0)
        {
            jumlah_genap++;
        }
    }

    printf("i            = %d\n", i);
    printf("total        = %d\n", total);
    printf("jumlah_genap = %d\n", jumlah_genap);

    return 0;

}
```

Screenshot Output Aktual

```
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrogram
an\Module_05\output> & .\'tracing.exe'
i            = 7
total        = 21
jumlah_genap = 3
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrogram
an\Module_05\output>
```

Bandingkan hasil aktual dengan hasil tracing.

Jawaban = hasil i berbeda dari prediksi awal, dan hal itu disebabkan karena data i terus ter-update ditambah 1. Dan saat i = 7 adalah false, maka tidak terhitung dalam output

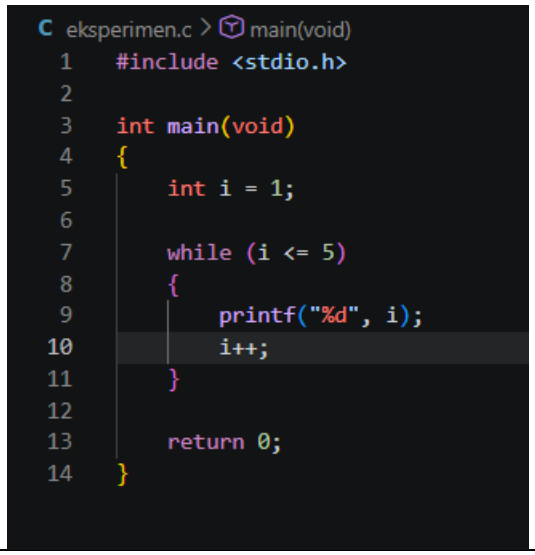
2. Eksperimen

Lakukan eksperimen satu per satu sesuai Modul 5. Eksperimen nomor 4 dan 5 berpotensi menyebabkan infinite loop; analisis terlebih dahulu sebelum menjalankannya.

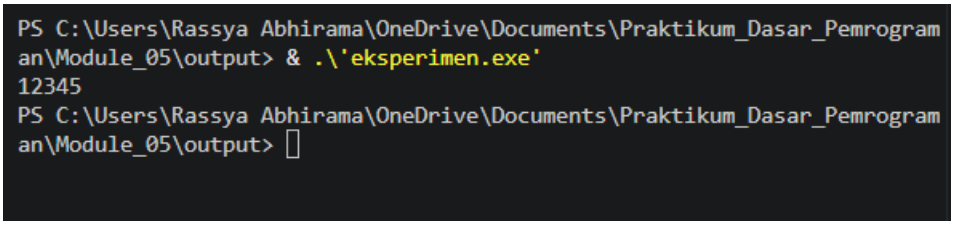
No.	Perubahan	Prediksi	Aman Dijalankan?	Hasil Aktual	Penjelasan
1	Ubah i <= 5 menjadi i < 5	Akan menampilkan 1 hingga 4	Ya	1234	Karena i < 5, maka nilai 5 tidak terbaca
2	Ubah nilai awal menjadi i = 0	Akan menampilkan 0 hingga 5	Ya	012345	Karena nilai awal Adalah 0, maka deretan akan dimulai dari 0-5
3	Ubah update menjadi i += 2	Akan menampilkan angka 1, 3, 5	Ya	135	Counter akan bertambah 2 secara konsisten
4	Ubah update menjadi i--	Akan terjadi loop	Tidak	Infinite loop	Karena i dikurang dan kondisi yang dijalankan adalah i <= 5, maka i akan terus dikurang tanpa batas
5	Ubah kondisi menjadi i >= 1 dengan update i++	Akan terjadi loop	tidak	Infinite loop	Karena i = 1, dan kondisi i >= 1 dengan update i++ maka program tidak memiliki batas update sehingga akan terjadi loop
6	Pindahkan printf() setelah perulangan	Akan menampilkan i = 6	Ya	6	Menampilkan nilai I setelah blok for Dimana iterasi berjalan hingga angka 6 = false
7	Tambahkan break ketika i == 3	1 2	Ya	12	Program berhenti Ketika i = 3
8	Tambahkan continue ketika i == 3	1 2 4 5	Ya	1245	Continue melewatkan pernyataan I == 3 dan meneruskan iterasi berikutnya

No.	Perubahan	Prediksi	Aman Dijalankan?	Hasil Aktual	Penjelasan
9	Ubah program menjadi while	12345	Ya	12345	While dan for memiliki fungsi yang mirip
10	Ubah program menjadi do-while	12345	Ya	12345	Karena perbedaan do-while bisa mengulang program jika blok salah

Screenshot Source Code Eksperimen yang Dipilih



Screenshot Output / Kondisi Program

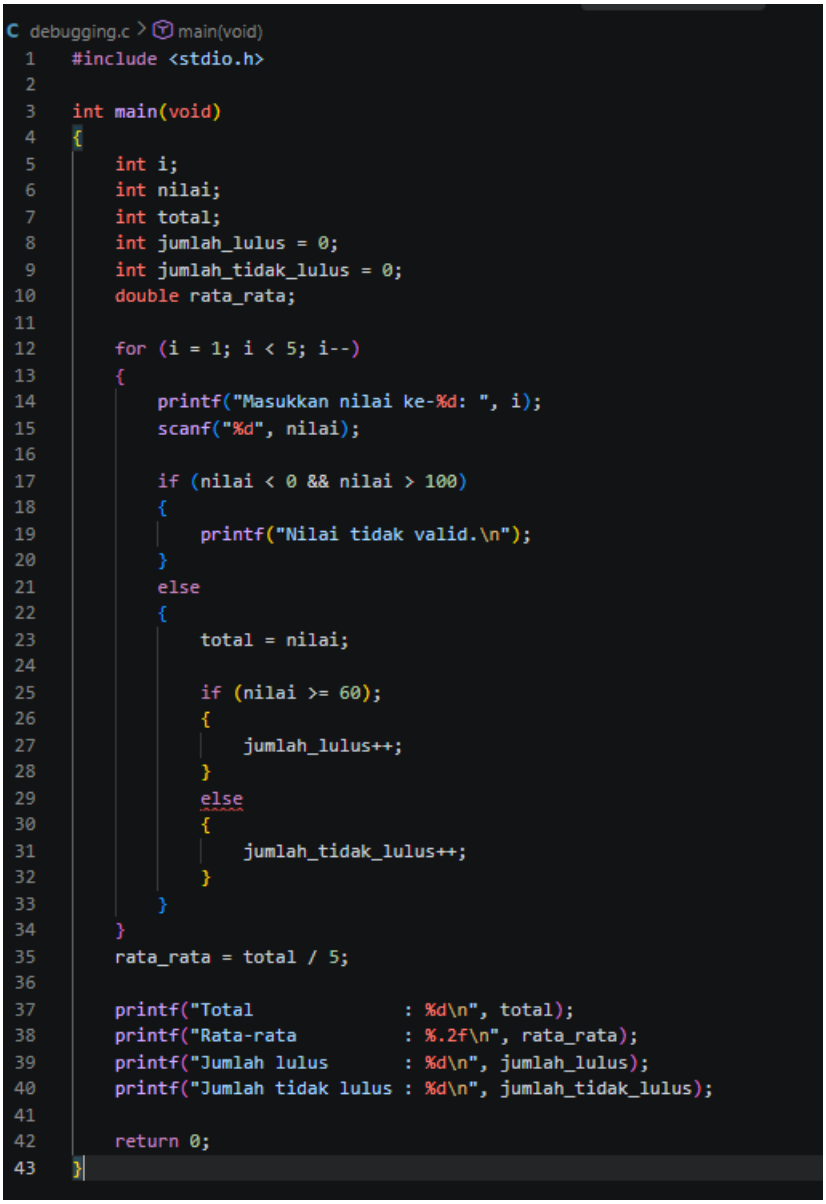


VI. Debugging Challenge

1. Debugging Challenge

Kerjakan berdasarkan program debugging.c pada Modul 5. Jangan langsung menjalankan program sebelum memeriksa risiko infinite loop.

Screenshot Source Code Sebelum Diperbaiki



```c
#include <stdio.h>

int main(void)
{
    int i;
    int nilai;
    int total;
    int jumlah_lulus = 0;
    int jumlah_tidak_lulus = 0;
    double rata_rata;

    for (i = 1; i < 5; i--)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%d", nilai);

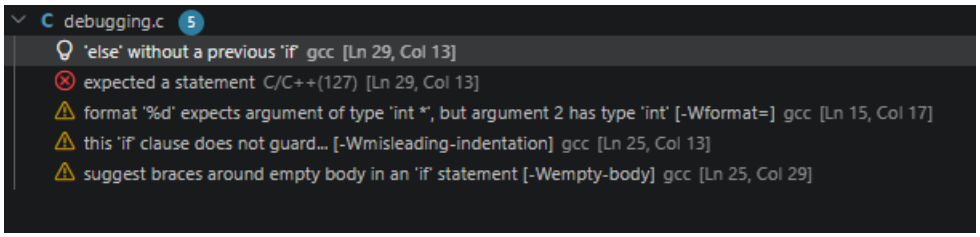
        if (nilai < 0 && nilai > 100)
        {
            printf("Nilai tidak valid.\n");
        }
        else
        {
            total = nilai;

            if (nilai >= 60);
            {
                jumlah_lulus++;
            }
            else
            {
                jumlah_tidak_lulus++;
            }
        }
    }
    rata_rata = total / 5;

    printf("Total              : %d\n", total);
    printf("Rata-rata          : %.2f\n", rata_rata);
    printf("Jumlah lulus       : %d\n", jumlah_lulus);
    printf("Jumlah tidak lulus : %d\n", jumlah_tidak_lulus);

    return 0;
}
```

Screenshot Hasil Kompilasi Awal (Error/Warning)

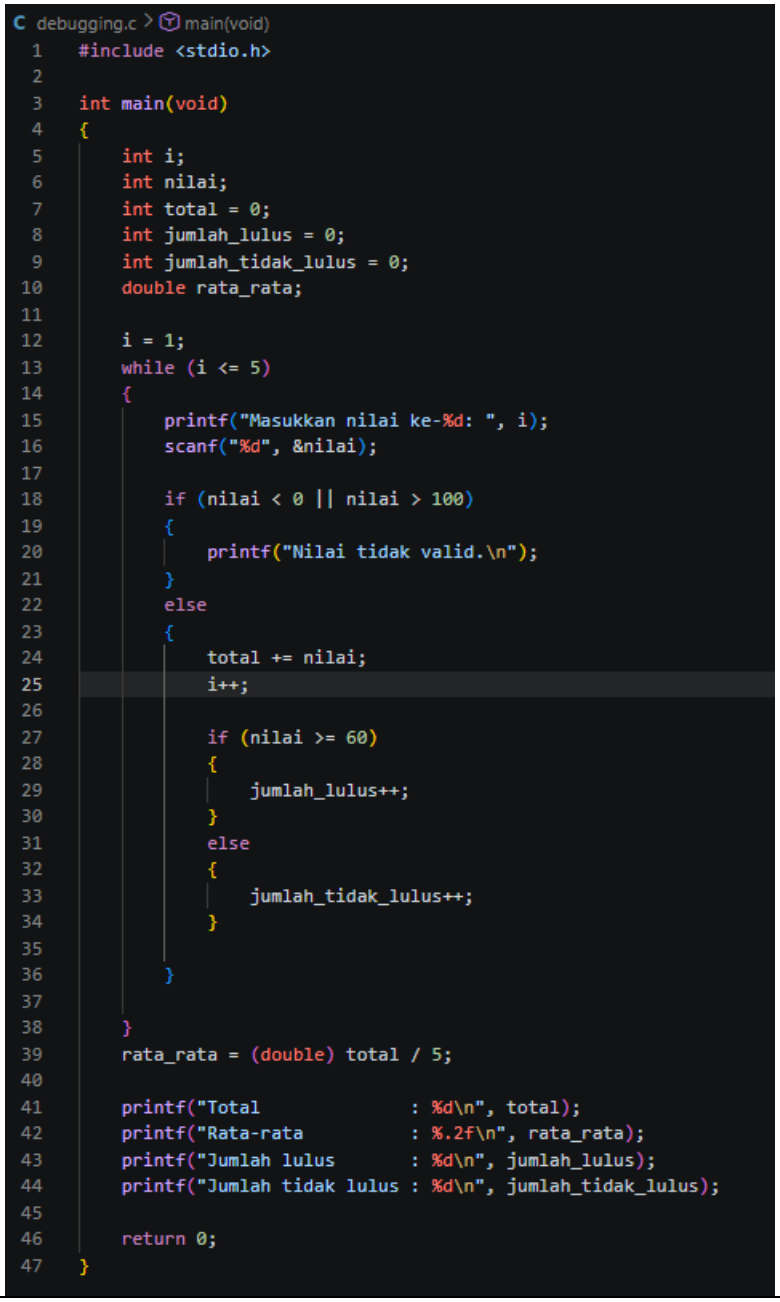


```
debugging.c 5
'else' without a previous 'if' gcc [Ln 29, Col 13]
expected a statement C/C++(127) [Ln 29, Col 13]
format '%d' expects argument of type 'int *', but argument 2 has type 'int' [-Wformat=] gcc [Ln 15, Col 17]
this 'if' clause does not guard... [-Wmisleading-indentation] gcc [Ln 25, Col 13]
suggest braces around empty body in an 'if' statement [-Wempty-body] gcc [Ln 25, Col 29]
```

Tabel Debugging

No.	Bagian yang Salah	Jenis Kesalahan	Penyebab	Perbaikan
1	Line 12	Logical error	i--	for (i = 1; i <= 5; i++)
2	Line 15	Syntax error	Kurang & pada scanf nilai	scanf("%d", &nilai);
3	Line 7	Syntax error	Nilai awal total tidak di inisialisasi	Int total = 0;
4	Line 17	Logical error	Karena memakai &&, angka berapapun akan salah	if (nilai < 0 || nilai > 100)
5	Line 23	Syntax error	Data total tidak ditambah oleh nilai, tetapi malah diubah dengan nilai berikutnya	total += nilai;
6	Line 25	Syntax error	Terdapat koma setelah if	If (nilai >= 60)
7	Line 35	Syntax error	Harus diubah mejadi tipe data float/double. Karena pada rumus yang dibagi Adalah tipe data int	rata_rata = (double) total / 5;
8	Tidak ada pengulangan pada kode	Logical error	Tidak ada while atau do-while	Ubah kode menjadi blok while sebelum printf() pertama

Screenshot Source Code Setelah Diperbaiki



```c
#include <stdio.h>

int main(void)
{
    int i;
    int nilai;
    int total = 0;
    int jumlah_lulus = 0;
    int jumlah_tidak_lulus = 0;
    double rata_rata;

    i = 1;
    while (i <= 5)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%d", &nilai);

        if (nilai < 0 || nilai > 100)
        {
            printf("Nilai tidak valid.\n");
        }
        else
        {
            total += nilai;
            i++;

            if (nilai >= 60)
            {
                jumlah_lulus++;
            }
            else
            {
                jumlah_tidak_lulus++;
            }

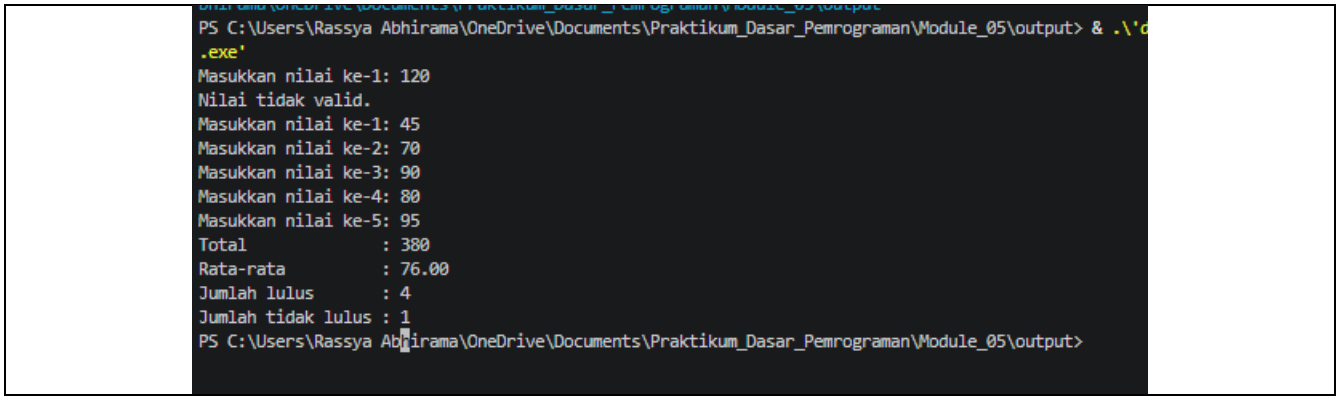
        }

    }
    rata_rata = (double) total / 5;

    printf("Total              : %d\n", total);
    printf("Rata-rata          : %.2f\n", rata_rata);
    printf("Jumlah lulus       : %d\n", jumlah_lulus);
    printf("Jumlah tidak lulus : %d\n", jumlah_tidak_lulus);

    return 0;
}
```

Screenshot Output Setelah Diperbaiki



Verifikasi hasil dengan lima nilai valid

Nilai 1	Nilai 2	Nilai 3	Nilai 4	Nilai 5	Total	Rata-rata	Lulus	Tidak Lulus
45	70	90	80	95	380	76.00	4	1

Jelaskan bagaimana perbaikan dilakukan agar program tetap meminta input sampai lima nilai valid berhasil diproses.

Jawaban = menambahkan while pada awal blok Dengan kondisi while (I <= 5) lalu buat perumpamaan jika nilai input tidak sesuai maka i tidak akan ditambah. Sehingga program akan terus mengulang hingga nilai yang di input benar dan barulah program akan menyimpan nilai i dan nilai data tersebut.

VII. Latihan Mandiri

1. Level 1 — Menampilkan Bilangan

Kerjakan sesuai Latihan Mandiri 1 pada Modul 5.

Screenshot Source Code

```c
#include <stdio.h>

int main(void)
{
    int batas_awal;
    int batas_akhir;
    int i;

    printf("Masukkan batas awal : ");
    scanf("%d", &batas_awal);
    printf("Masukkan batas akhir: ");
    scanf("%d", &batas_akhir);

    if (batas_awal > batas_akhir)
    {
        printf("Batas awal harus lebih kecil atau sama dengan batas akhir.\n");
        return 1;
    }

    for (i = batas_awal; i <= batas_akhir; i++)
    {
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

Screenshot Output

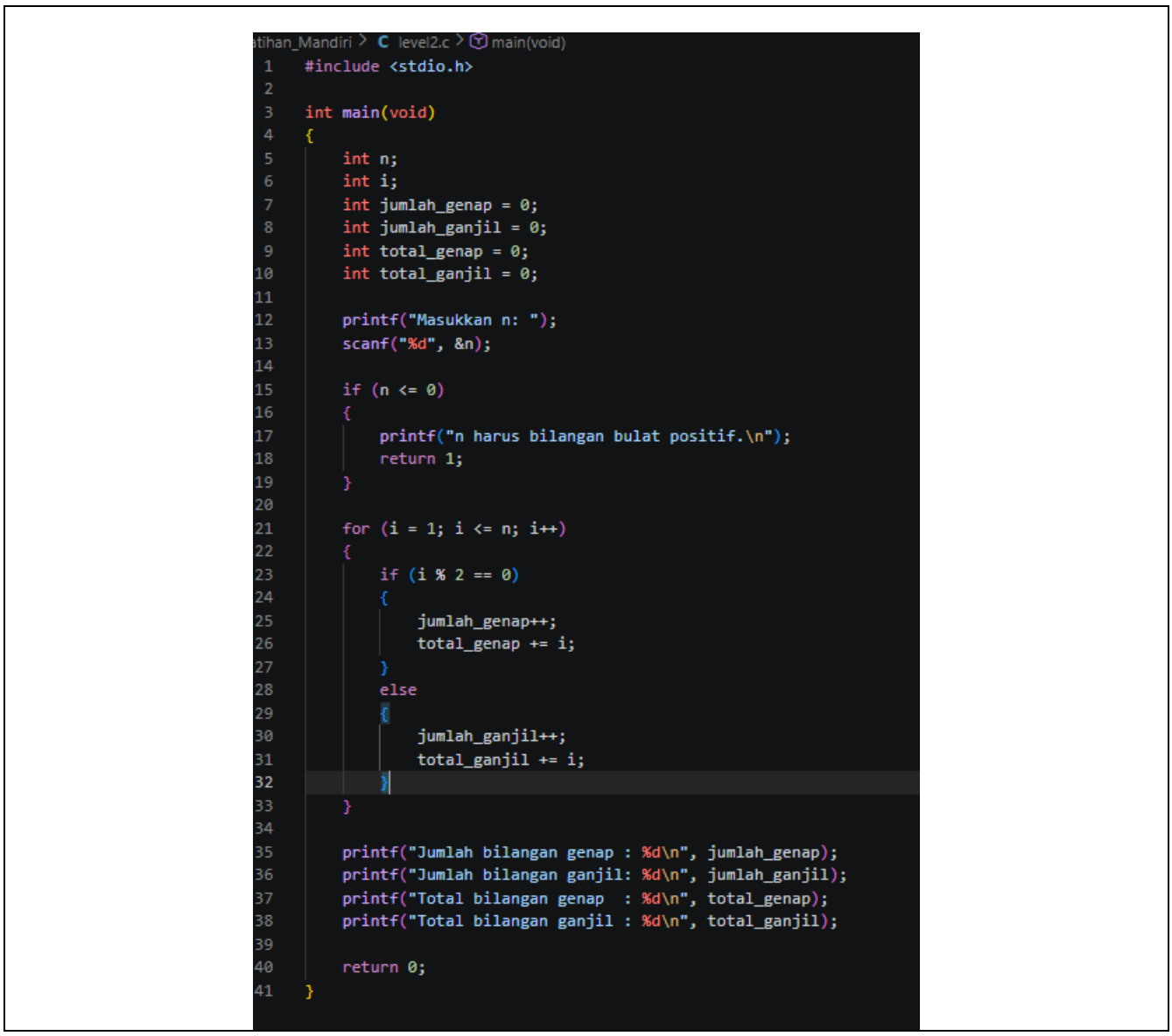
```
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mand
> & .\'level01.exe'
Masukkan batas awal : 3
Masukkan batas akhir: 8
3 4 5 6 7 8
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mand
> & .\'level01.exe'
Masukkan batas awal : 8
Masukkan batas akhir: 3
Batas awal harus lebih kecil atau sama dengan batas akhir.
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mand
>
```

Test Case	Batas Awal	Batas Akhir	Expected Output	Actual Output	Status
1	3	8	345678	345678	Berhasil
2	5	5	55	5	Berhasil
3	8	3	Batas awal invalid	Batas awal harus lebih kecil dari batas akhir	Berhasil

2. Level 2 — Menghitung Bilangan Genap dan Ganjil

Kerjakan sesuai Latihan Mandiri 2 pada Modul 5.

Screenshot Source Code



```
tihan_Mandiri > C level2.c > main(void)
#include <stdio.h>

int main(void)
{
    int n;
    int i;
    int jumlah_genap = 0;
    int jumlah_ganjil = 0;
    int total_genap = 0;
    int total_ganjil = 0;

    printf("Masukkan n: ");
    scanf("%d", &n);

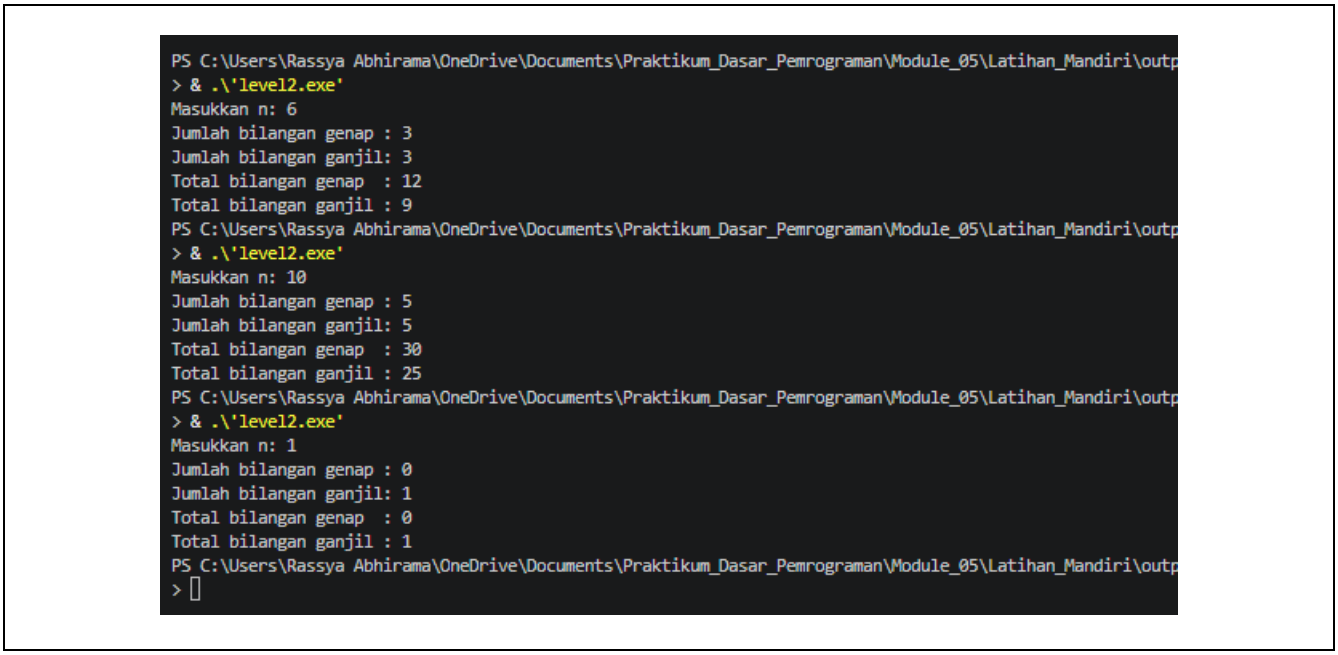
    if (n <= 0)
    {
        printf("n harus bilangan bulat positif.\n");
        return 1;
    }

    for (i = 1; i <= n; i++)
    {
        if (i % 2 == 0)
        {
            jumlah_genap++;
            total_genap += i;
        }
        else
        {
            jumlah_ganjil++;
            total_ganjil += i;
        }
    }

    printf("Jumlah bilangan genap : %d\n", jumlah_genap);
    printf("Jumlah bilangan ganjil: %d\n", jumlah_ganjil);
    printf("Total bilangan genap  : %d\n", total_genap);
    printf("Total bilangan ganjil : %d\n", total_ganjil);

    return 0;
}
```

Screenshot Output



n	Jumlah Genap	Jumlah Ganjil	Total Genap	Total Ganjil
6	3	3	12	9
10	5	5	30	25
1	0	1	0	1

3. Level 3 — Menghitung Faktorial

Kerjakan sesuai Latihan Mandiri 3 pada Modul 5. Perhatikan batas input 0–20 dan gunakan tipe unsigned long long.

Screenshot Source Code

Latihan_Mandiri > C level3.c > main(void)

```c
#include <stdio.h>

int main(void)
{
    int n;
    int i;
    unsigned long long faktorial = 1;

    printf("Masukkan bilangan 0-20: ");
    scanf("%d", &n);

    if (n < 0 || n > 20)
    {
        printf("Input tidak valid.\n");
        return 1;
    }

    for (i = 1; i <= n; i++)
    {
        faktorial *= i;
    }

    printf("%d! = %llu\n", n, faktorial);

    return 0;
}
```

Screenshot Output

```
> & .\'level3.exe'
Masukkan bilangan 0-20: 1
1! = 1
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mandiri\outp
> & .\'level3.exe'
Masukkan bilangan 0-20: 0
0! = 1
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mandiri\outp
> & .\'level3.exe'
Masukkan bilangan 0-20: 5
5! = 120
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mandiri\outp
> & .\'level3.exe'
Masukkan bilangan 0-20: 20
20! = 2432902008176640000
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mandiri\outp
> & .\'level3.exe'
Masukkan bilangan 0-20: 21
Input tidak valid.
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\Latihan_Mandiri\outp
>
```

Test Case	Input	Expected Output	Actual Output	Status
1	0	0! = 1	1	Berhasil

Test Case	Input	Expected Output	Actual Output	Status
2	1	1! = 1	1	Berhasil
3	5	5! = 120	5! = 120	Berhasil
4	20	20! = 2432902008176640000	20! = 2432902008176640000	Berhasil
5	21	Input tidak valid	Input tidak valid	Berhasil

4. Challenge — Membentuk Bilangan Terbalik

Kerjakan sesuai Challenge pada Modul 5 menggunakan while.

Screenshot Source Code

```c
#include <stdio.h>

int main(void)
{
    long bilangan;
    long asli;
    long hasil = 0;
    int digit;

    printf("Masukkan bilangan: ");
    scanf("%ld", &bilangan);

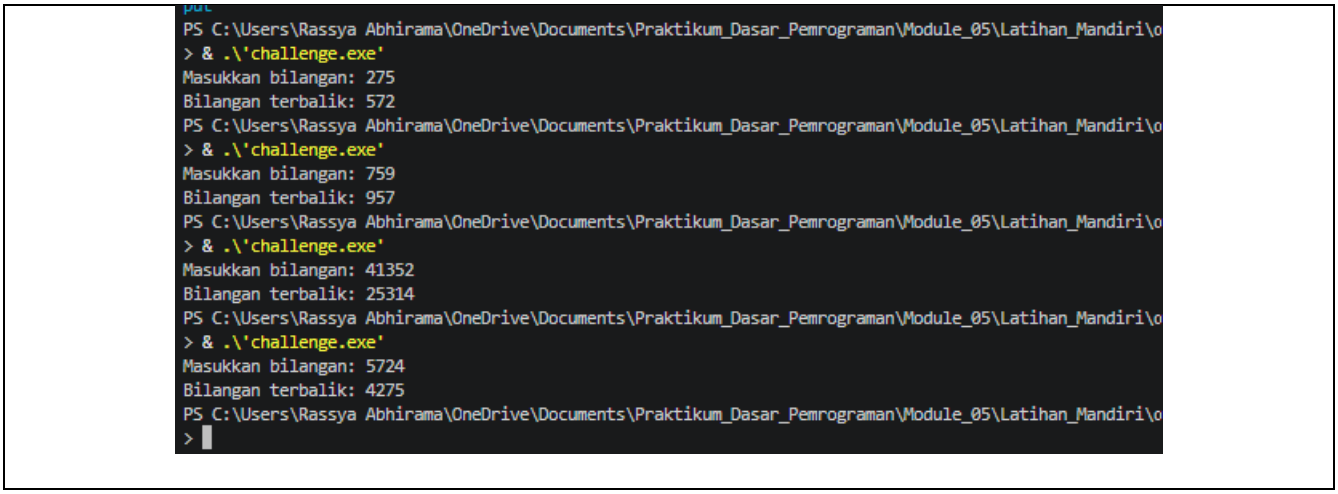
    asli = bilangan;

    while (bilangan > 0)
    {
        digit = bilangan % 10;
        hasil = hasil * 10 + digit;
        bilangan = bilangan / 10;
    }

    printf("Bilangan terbalik: %ld\n", hasil);
    (void) asli;

    return 0;
}
```

Screenshot Output



Tracing untuk bilangan 572

Iterasi	Bilangan	Digit	Hasil
Awal	572	–	0
1	57	2	2
2	5	7	27
3	0	5	275

5. Jumlah Digit Bilangan

Kerjakan sesuai Latihan Mandiri 5 pada Modul 5. Perhatikan bahwa output diperiksa secara ketat dan label “Jumlah Digit” berarti jumlah total seluruh digit.

Screenshot Source Code

Latihan_Mandiri > C digit_bilangan.c > main(void)

```c
#include <stdio.h>

int main(void)
{
    int bilangan;
    int sisa;
    int jumlah = 0;

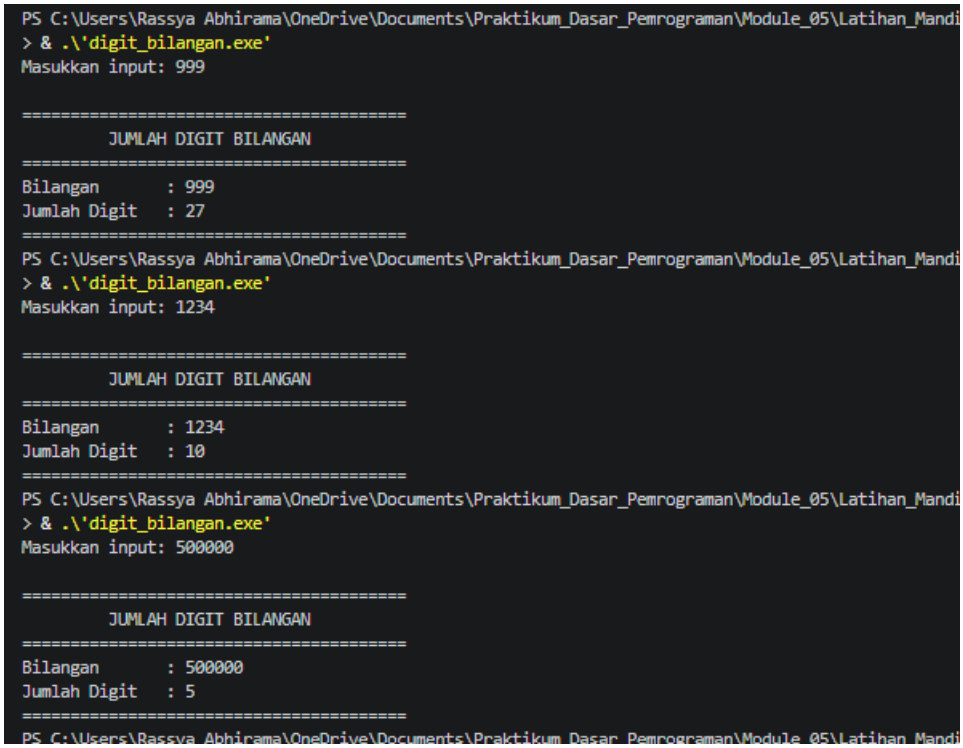
    printf("Masukkan input: ");
    scanf("%d", &bilangan);

    sisa = bilangan;
    while (sisa > 0)
    {
        jumlah += sisa % 10;
        sisa = sisa / 10;
    }

    printf("\n");
    printf("========================================\n");
    printf("         JUMLAH DIGIT BILANGAN          \n");
    printf("========================================\n");
    printf("Bilangan       : %d\n", bilangan);
    printf("Jumlah Digit   : %d\n", jumlah);
    printf("========================================\n");

    return 0;
}
```

Screenshot Output



Test Case	Input	Expected Jumlah Digit	Actual Output	Status
1	1234	10	10	Berhasil
2	999	27	27	Berhasil
3	500000	5	5	Berhasil

VIII. Pengujian Program

1. Pengujian Program Perulangan

Gunakan program pada VII.3 Level 3 — Menghitung Faktorial. Jalankan program dengan setiap input pada tabel berikut, tuliskan Actual Output, lalu tentukan Status dengan membandingkannya terhadap Expected Output.

No.	Input	Expected Output	Actual Output	Status
1	-1	Input tidak valid	Input tidak valid.	Berhasil
2	0	1	1	Berhasil
3	1	1	1	Berhasil
4	5	120	120	Berhasil

No.	Input	Expected Output	Actual Output	Status
5	10	3.628.800	3628800	Berhasil
6	20	2.432.902.008.176.640.000	2432902008176640000	Berhasil
7	21	Input tidak valid	Input tidak valid.	Berhasil

Catatan pengujian kondisi berhenti / risiko infinite loop:

Jawaban = program tidak akan mengalami Risiko infinite loop asal kondisi i <= n pada for tidak berubah.

IX. Tugas Praktikum

1. Tugas Praktikum Modul 5 — Program Rekap Nilai Mahasiswa

Kerjakan tugas utama sesuai ketentuan pada Modul 5. Source code tetap diketik sendiri dan tidak ditulis ulang pada LKP.

6. Analisis IPO

Komponen	Uraian
Input	Jumlah mahasiswa, nilai mahasiswa
Process	Validasi jumlah mahasiswa, validasi nilai mahasiswa, menghitung total nilai, menghitung jumlah lulus dan jumlah tidak lulus dengan counter, menentukan nilai tertinggi dan terendah, dan yang terakhir menghitung persentase kelulusan
Output	Hasil Rekap Nilai Mahasiswa

7. Daftar Variabel

Variabel	Tipe Data	Nilai Awal	Fungsi
jumlah_mahasiswa	Int	-	Menentukan berapa input yang ingin diisi
i	Int	1	Menentukan berapa iterasi yang ingin dijalankan
nilai	Double	-	Menyatakan nilai tiap mahasiswa
total_nilai	Double	0	Menghitung jumlah seluruh nilai-nilai mahasiswa
jumlah_lulus	Int	0	Menentukan berapa mahasiswa yang diatas kkm

Variabel	Tipe Data	Nilai Awal	Fungsi
jumlah_tidak_lulus	Int	0	Menentukan ada berapa mahasiswa yang dibawah kkm
nilai_tertinggi	Double	0	Menentukan nilai tertinggi dari seluruh input nilai
nilai_terendah	Double	0	Menentukan nilai terendah dari seluruh input nilai
rata_rata	Double	-	Menghitung rata rata nilai mahasiswa
persentase_lulus	Double	-	Menghitung persentase kelulusan

Pseudocode atau Flowchart

Pseudocode / Flowchart

```
pseudocode
START

    input jumlah mahasiswa
    input nilai masing-masing mahasiswa

    menghitung total nilai;
    menghitung rata-rata = total nilai / jumlah mahasiswa;
    menentukan nilai tertinggi;
    menentukan nilai terendah;
    menentukan jumlah lulus if (nilai >= 60)
    menentukan jumlah tidak lulus if (nilai <= 60)
    menghitung persentase kelulusan = jumlah lulus / jumlah mahasiswa

    output hasil rekapan nilai mahasiswa

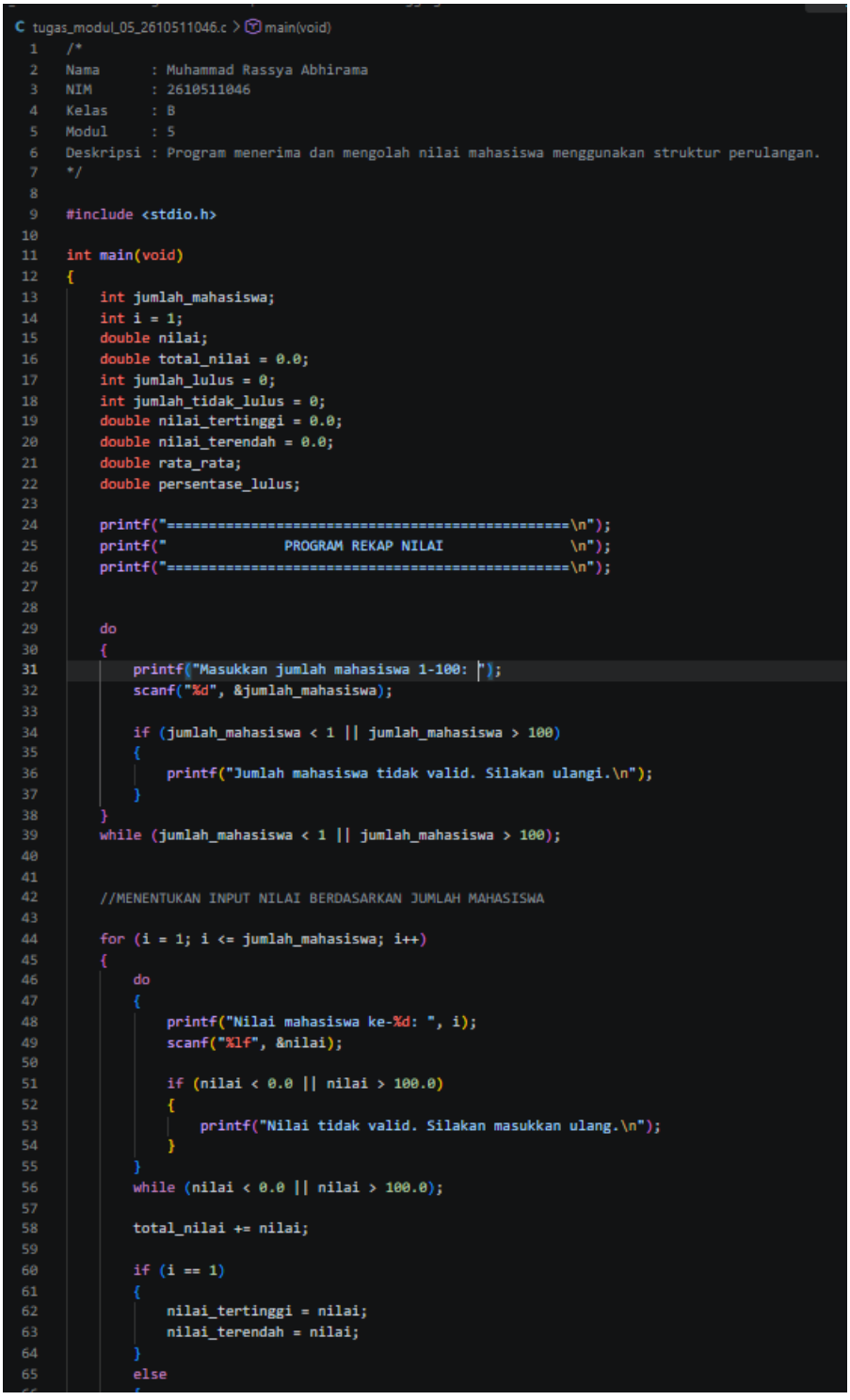
END
```

Trace table untuk satu data uji

Langkah/Iterasi	Nilai	Total	Tertinggi	Terendah	Lulus	Tidak Lulus
1 / input jumlah mahasiswa	-	0	-	-	0	0
2 / mahasiswa ke-1	80	80	80	80	1	0

Langkah/Iterasi	Nilai	Total	Tertinggi	Terendah	Lulus	Tidak Lulus
3 / mahasiswa ke-2	59	139	80	59	1	1
4 / mahasiswa ke-3	85	224	85	59	2	1
5 / mahasiswa ke-4	60	284	85	59	3	1
6 / mahasiswa ke-5	75	359	85	59	4	1
7 / rata-rata	71.80	359	85	59	4	1
8 / persentase	80.00%	359	85	59	4	1

Screenshot Source Code tugas_modul_05_nim.c



```c
/*
Nama      : Muhammad Rassya Abhirama
NIM       : 2610511046
Kelas     : B
Modul     : 5
Deskripsi : Program menerima dan mengolah nilai mahasiswa menggunakan struktur perulangan.
*/

#include <stdio.h>

int main(void)
{
    int jumlah_mahasiswa;
    int i = 1;
    double nilai;
    double total_nilai = 0.0;
    int jumlah_lulus = 0;
    int jumlah_tidak_lulus = 0;
    double nilai_tertinggi = 0.0;
    double nilai_terendah = 0.0;
    double rata_rata;
    double persentase_lulus;

    printf("================================================\n");
    printf("              PROGRAM REKAP NILAI               \n");
    printf("================================================\n");


    do
    {
        printf("Masukkan jumlah mahasiswa 1-100: ");
        scanf("%d", &jumlah_mahasiswa);

        if (jumlah_mahasiswa < 1 || jumlah_mahasiswa > 100)
        {
            printf("Jumlah mahasiswa tidak valid. Silakan ulangi.\n");
        }
    }
    while (jumlah_mahasiswa < 1 || jumlah_mahasiswa > 100);


    //MENENTUKAN INPUT NILAI BERDASARKAN JUMLAH MAHASISWA

    for (i = 1; i <= jumlah_mahasiswa; i++)
    {
        do
        {
            printf("Nilai mahasiswa ke-%d: ", i);
            scanf("%lf", &nilai);

            if (nilai < 0.0 || nilai > 100.0)
            {
                printf("Nilai tidak valid. Silakan masukkan ulang.\n");
            }
        }
        while (nilai < 0.0 || nilai > 100.0);

        total_nilai += nilai;

        if (i == 1)
        {
            nilai_tertinggi = nilai;
            nilai_terendah = nilai;
        }
        else
```

```
        else
        {
            if (nilai > nilai_tertinggi)
            {
                nilai_tertinggi = nilai;
            }
            if (nilai < nilai_terendah)
            {
                nilai_terendah = nilai;
            }
        }

        if (nilai >= 60.0)
        {
            jumlah_lulus++;
        }
        else
        {
            jumlah_tidak_lulus++;
        }
    }

    rata_rata = total_nilai / jumlah_mahasiswa;
    persentase_lulus = (double) jumlah_lulus / jumlah_mahasiswa * 100.0;

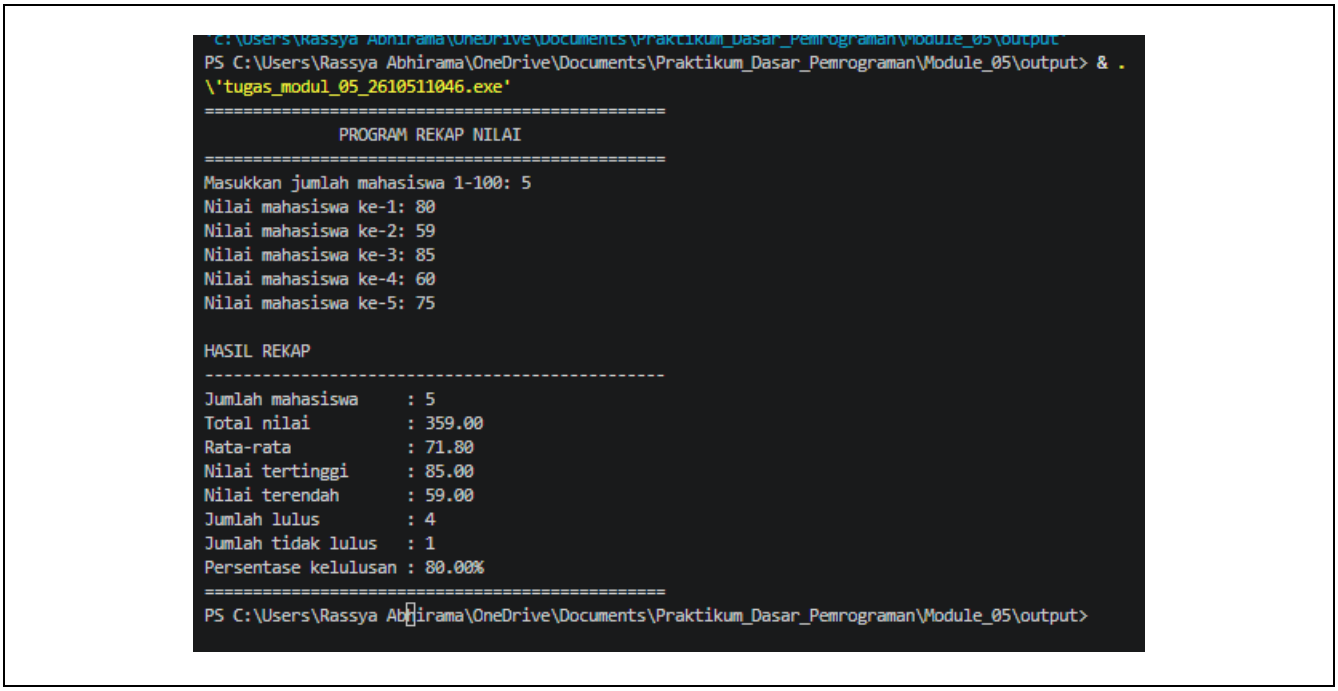
    printf("\nHASIL REKAP\n");
    printf("-----------------------------------------------\n");
    printf("Jumlah mahasiswa     : %d\n", jumlah_mahasiswa);
    printf("Total nilai          : %.2f\n", total_nilai);
    printf("Rata-rata            : %.2f\n", rata_rata);
    printf("Nilai tertinggi      : %.2f\n", nilai_tertinggi);
    printf("Nilai terendah       : %.2f\n", nilai_terendah);
    printf("Jumlah lulus         : %d\n", jumlah_lulus);
    printf("Jumlah tidak lulus   : %d\n", jumlah_tidak_lulus);
    printf("Persentase kelulusan : %.2f%%\n", persentase_lulus);
    printf("===============================================\n");

    return 0;
}
```

Screenshot Proses Kompilasi dengan Warning Aktif

```
===============================================
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\output> cd
'c:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\output'
PS C:\Users\Rassya Abhirama\OneDrive\Documents\Praktikum_Dasar_Pemrograman\Module_05\output> & .
\'tugas_modul_05_2610511046.exe'
===============================================
              PROGRAM REKAP NILAI
```

Screenshot Hasil Eksekusi



Tabel pengujian minimal delapan kasus

No.	Jumlah Mahasiswa	Data Nilai	Expected Result Utama	Actual Result	Status
1	0	–	Input jumlah ditolak	Tidak valid	Berhasil
2	101	–	Input jumlah ditolak	Tidak valid	Berhasil
3	1	60	Rata-rata 60, lulus 1	Rata-rata = 60 Jumlah lulus = 1	Berhasil
4	1	0	Rata-rata 0, tidak lulus 1	Rata-rata = 0 Jumlah tidak lulus = 0	Berhasil
5	3	50, 60, 70	Rata-rata 60, lulus 2	Rata-rata = 60 Lulus = 2	Berhasil
6	4	100, 100, 100, 100	Rata-rata 100	Rata-rata = 100	Berhasil
7	5	80, 55, 70, 90, 60	Rata-rata 71, lulus 4	Rata-rata = 71 Lulus = 4	Berhasil
8	3	-1, 50, 101, 60, 70	Input tidak valid diminta ulang	Program terus berjalan, tetapi angka yang tidak valid tidak terhitung	Berhasil

No.	Jumlah Mahasiswa	Data Nilai	Expected Result Utama	Actual Result	Status
9	3	0, 0, 0	Tertinggi dan terendah 0	Tertinggi = 0 Tertinggi = 0	Berhasil
10	3	100, 0, 50	Tertinggi 100, terendah 0	Tertinggi = 100 Terendah = 0	Berhasil

Refleksi

1. Struktur perulangan mana yang paling banyak digunakan pada tugas ini, dan mengapa?

Jawaban = struktur while karena banyak program yang menggunakan system validasi nilai

2. Bagaimana program memastikan hanya nilai valid yang dihitung?

Jawaban = menggunakan do-while untuk mengulang program sehingga kondisi tertentu tercapai

3. Kesalahan apa yang paling sulit Anda temukan atau hindari pada tugas ini?

Jawaban = saya sering mendapat kesalahan Ketika membuat struktur while, karena sering tertukar dengan do-while

3. Simulasi Tabungan Berbunga

Kerjakan tugas tambahan sesuai Modul 5 dan simpan sebagai tugas_modul_05_soal2_nim.c. Perhatikan bahwa format output diperiksa secara ketat.

Screenshot Source Code

tugas_modul_05_soal2_2610511046.c > main(void)

```c
int main(void)
{
    double saldo;
    double suku_bunga;
    int jumlah_bulan;
    int bulan;

    printf("Saldo awal: ");
    scanf("%lf", &saldo);
    printf("Bunga/bulan: ");
    scanf("%lf", &suku_bunga);
    printf("Jumlah bulan: ");
    scanf("%d", &jumlah_bulan);

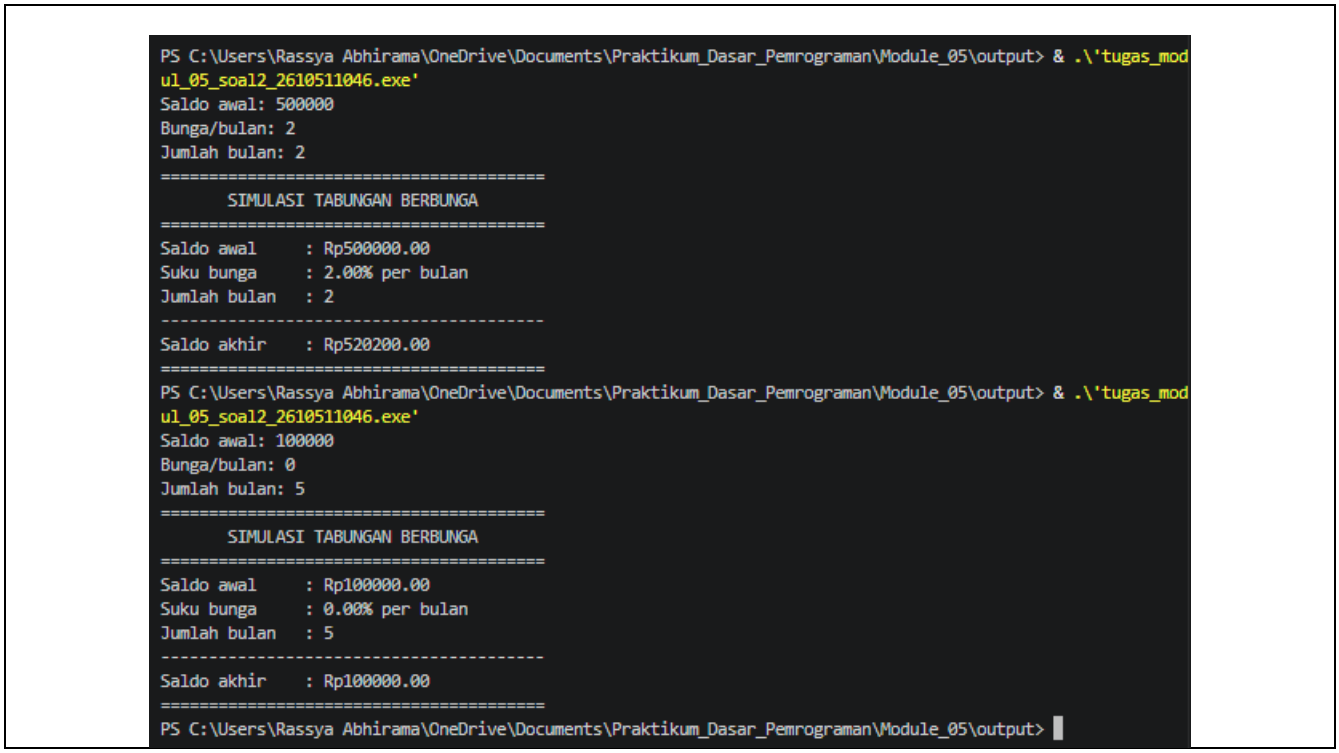
    printf("========================================\n");
    printf("       SIMULASI TABUNGAN BERBUNGA       \n");
    printf("========================================\n");
    printf("Saldo awal     : Rp%.2f\n", saldo);
    printf("Suku bunga     : %.2f%% per bulan\n", suku_bunga);
    printf("Jumlah bulan   : %d\n", jumlah_bulan);
    printf("----------------------------------------\n");

    for (bulan = 1; bulan <= jumlah_bulan; bulan++)
    {
        saldo = saldo + (saldo * suku_bunga / 100);
    }

    printf("Saldo akhir    : Rp%.2f\n", saldo);
    printf("========================================\n");

    return 0;
}
```

Screenshot Output



Test Case	Saldo Awal	Bunga/Bulan	Jumlah Bulan	Expected Saldo Akhir	Actual Output	Status
1	1000000	1	3	Rp1030301.00	Rp1030301.00	Berhasil
2	500000	2	2	Rp520200.00	Rp520200.00	Berhasil
3	100000	0	5	Rp100000.00	Rp100000.00	Berhasil

Jelaskan mengapa saldo bulan berikutnya harus dihitung dari saldo hasil bulan sebelumnya, bukan selalu dari saldo awal.

Jawaban = karena dalam dunia perbankan, suku bunga itu dihitung dari saldo terkini sehingga kita harus menggunakan struktur for untuk menyesuaikan jumlah bulan

XII. Pernyataan Penggunaan AI

Pilih dan lengkapi salah satu pernyataan berikut.

Menggunakan AI: Saya menggunakan gemini untuk membantu menjelaskan dalam perbedaan struktur do-while, while, dan for. Saya juga menggunakan gemini untuk membantu dalam logical concept beberapa program.

Hasil tersebut saya periksa melalui tracing, kompilasi, pengujian kondisi batas, dan perbandingan dengan algoritma yang telah disusun:

Semua program saya kompilasi ulang dan saya cek Kembali dengan modul praktikum untuk menyesuaikan seluruh kode sama seperti modul. Dan pada tracing table saya sesuaikan dengan uji coba program.